

UniApp【萌宠案例】通过请求头传递access-key的API调用实现

在萌宠案例的API调用中，部分接口可能需要通过请求头携带access-key进行权限验证，以确保接口访问安全。本方案将在原有请求工具类基础上，新增access-key的获取、存储与请求头注入逻辑，同步适配首页、分类页、详情页的所有API调用，同时解决多端存储兼容性与密钥更新问题，保证接口调用的合法性与稳定性。

一、核心原理：access-key的作用与传递规则

access-key是服务端分配给客户端的身份标识，用于验证请求来源的合法性，通常由后端开发人员提供。通过请求头传递access-key的核心规则如下：

- 传递位置：**统一在HTTP请求头中携带，键名通常为 `access-key`（需与后端约定，如无特殊说明则使用此键名）。
- 存储方式：**在UniApp中使用 `uni.getStorageSync` / `uni.setStorageSync` 存储，确保多页面共享且持久化（除非主动清除）。
- 优先级：**支持页面级临时覆盖（如部分接口使用特殊密钥），但默认优先使用全局存储的access-key。
- 异常处理：**若access-key缺失或无效，服务端会返回403权限错误，需在请求拦截中统一捕获并引导用户配置。



实际开发中，access-key需妥善保管，避免明文硬编码在代码中。对于需要动态获取密钥的场景（如通过登录接口返回），可扩展本方案的密钥获取逻辑。

二、基础配置：access-key的存储与初始化

首先实现access-key的存储初始化功能，提供全局配置入口与页面级调用方法，确保密钥可灵活管理。

1. 全局配置文件（utils/config.js）

定义access-key的默认存储键名与接口相关配置，便于后续维护与修改。

```

1 // 1. 全局配置
2 * 全局配置文件
3 * 包含access-key存储配置、API基础信息等
4 */
5 export default {
6   // access-key相关配置
7   accessKey: {
8     storageKey: 'pet_case_access_key', // 存储在本地的键名
9     defaultKey: '' // 开发环境默认密钥（生产环境需移除，通过用户配置或接口获取）
10  },
11   // API基础配置
12   api: {
13     baseUrl: 'https://api.pet.com', // 基础API地址
14     timeout: 10000 // 默认超时时间
15  },
16   // 错误码配置
17   errorCode: {
18     noAccessKey: 10001, // 本地无access-key
19     invalidAccessKey: 403 // 服务端返回的密钥无效错误码
20  }
21 }

```

2. access-key工具类（utils/accessKeyUtil.js）

封装access-key的获取、存储、清除方法，统一处理多端存储差异，提供便捷的调用接口。

Code block

```

1 import config from './config'
2
3 /**
4  * access-key工具类
5  * 负责密钥的存储、获取与验证
6  */
7 const accessKeyUtil = {
8   /**
9    * 存储access-key到本地
10   * @param {string} key - 要存储的access-key
11   * @returns {boolean} - 存储是否成功
12   */
13   setAccessKey(key) {
14     if (!key || typeof key !== 'string') {
15       console.error('access-key格式错误，必须为非空字符串')
16       return false
17     }
18     try {
19       uni.setStorageSync(config.accessKey.storageKey, key)

```

```
20     return true
21   } catch (err) {
22     console.error('access-key存储失败: ', err)
23     return false
24   }
25 },
26
27 /**
28  * 从本地获取access-key
29  * @returns {string} - 获取到的access-key (为空则返回默认值)
30  */
31 getAccessKey() {
32   try {
33     const storedKey = uni.getStorageSync(config.accessKey.storageKey)
34     // 优先使用本地存储的密钥，若无则返回默认密钥
35     return storedKey || config.accessKey.defaultKey
36   } catch (err) {
37     console.error('access-key获取失败: ', err)
38     return config.accessKey.defaultKey
39   }
40 },
41
42 /**
43  * 清除本地存储的access-key
44  * @returns {boolean} - 清除是否成功
45  */
46 clearAccessKey() {
47   try {
48     uni.setStorageSync(config.accessKey.storageKey, '')
49     return true
50   } catch (err) {
51     console.error('access-key清除失败: ', err)
52     return false
53   }
54 },
55
56 /**
57  * 验证access-key是否有效 (本地验证, 非服务端验证)
58  * @returns {boolean} - 密钥是否有效
59  */
60 validateAccessKey() {
61   const key = this.getAccessKey()
62   // 简单验证: 非空且长度符合常规密钥长度 (如16-32位, 可根据实际调整)
63   return !!key && (key.length >= 16 && key.length <= 32)
64 }
65 }
66
```

三、核心修改：请求工具类注入access-key

这是本次改造的核心部分，在原有请求工具类中集成access-key的请求头注入逻辑，同时处理密钥缺失、无效等异常场景。

请求工具类（utils/request.js）

Code block

```

1  import config from './config'
2  import accessKeyUtil from './accessKeyUtil'
3
4  /**
5   * 通用请求工具类
6   * 新增access-key请求头注入功能，统一处理权限验证异常
7   * @param {Object} options - 请求配置
8   * @returns {Promise} - 响应结果
9   */
10 const request = (options) => {
11   // 1. 基础配置合并
12   const baseConfig = {
13     url: config.api.baseUrl + options.url,
14     method: options.method || 'GET',
15     timeout: options.timeout || config.api.timeout,
16     sslVerify: options.sslVerify !== false,
17     showLoading: options.showLoading !== false // 是否显示加载动画，默认显示
18   }
19
20   // 2. 处理请求头：注入access-key
21   const headers = {
22     'Content-Type': 'application/json',
23     // 优先使用options.header中的access-key（页面级覆盖），否则使用全局密钥
24     'access-key': options.header?.['access-key'] ||
25     accessKeyUtil.getAccessKey(),
26     ...options.header // 允许页面级覆盖其他请求头
27   }
28
29   // 3. 验证access-key是否存在（本地预验证，减少无效请求）
30   if (!headers['access-key'] && !options.ignoreAccessKey) {
31     // 若接口未设置忽略密钥，且密钥缺失，直接触发异常
32     return new Promise((_, reject) => {
33       const errMsg = 'access-key缺失，请先配置密钥'
34       uni.showToast({ title: errMsg, icon: 'error' })
35     })
36   }
37 }

```

```

34     // 引导跳转到密钥配置页（需自行创建配置页面）
35     setTimeout(() => {
36         uni.navigateTo({ url: '/pages/setting/accessKeySetting' })
37     }, 1500)
38     reject(new Error(errMsg))
39 })
40 }
41
42 // 4. 发起请求并处理结果
43 return new Promise((resolve, reject) => {
44     // 显示加载动画
45     if (baseConfig.showLoading) {
46         uni.showLoading({ title: '加载中...', mask: true })
47     }
48
49     uni.request({
50         ...baseConfig,
51         header: headers,
52         data: options.data || {},
53         // 成功回调：处理响应与权限异常
54         success: (res) => {
55             const { statusCode, data } = res
56
57             // 处理服务端返回的密钥无效错误（403或自定义错误码）
58             if (statusCode === config.errorCode.invalidAccessKey || data.code ===
config.errorCode.invalidAccessKey) {
59                 const errMsg = 'access-key无效，请重新配置'
60                 uni.showToast({ title: errMsg, icon: 'error' })
61                 // 清除无效密钥并跳转到配置页
62                 accessKeyUtil.clearAccessKey()
63                 setTimeout(() => {
64                     uni.navigateTo({ url: '/pages/setting/accessKeySetting' })
65                 }, 1500)
66                 reject(new Error(errMsg))
67                 return
68             }
69
70             // 正常处理HTTP与业务状态码
71             if (statusCode >= 200 && statusCode < 300) {
72                 if (data.code === 0) {
73                     resolve(data.data)
74                 } else {
75                     uni.showToast({ title: data.msg || '请求失败', icon: 'error' })
76                     reject(new Error(data.msg || '请求失败'))
77                 }
78             } else {
79                 const errMsg = `请求错误[${statusCode}]: ${data.msg || '服务器异常'}`

```

```

80         uni.showToast({ title: errMsg, icon: 'error' })
81         reject(new Error(errMsg))
82     }
83 },
84 // 失败回调：网络错误等
85 fail: (err) => {
86     // 网络错误提示
87     uni.showToast({ title: '网络错误', icon: 'error' })

```

四、页面适配：密钥配置页与业务页面调用

新增access-key配置页面，用于用户手动输入密钥，同时确保原有首页、分类页、详情页的API调用无需额外修改（因请求工具类已自动注入密钥）。

1. access-key配置页面（pages/setting/accessKeySetting.vue）

提供密钥输入、保存功能，实时验证密钥格式，确保存储的密钥有效。

Code block

```

1  <template>
2    <view class="access-key-page">
3      <view class="page-header">
4        <text class="page-title">接口密钥配置</text>
5        <text class="page-desc">请输入有效的access-key以访问萌宠接口</text>
6      </view>
7
8      <view class="form-container pet-card">
9        <view class="form-item">
10         <text class="label">access-key</text>
11         <input
12           class="input"
13           type="text"
14           v-model="accessKey"
15           placeholder="请输入16-32位密钥"
16           @input="validateInput"
17         />
18       </view>
19       <text class="tip" :class="{ valid: isKeyValid, invalid: !isKeyValid &&
20         accessKey.length }">
21         {{ isKeyValid ? '密钥格式有效' : '请输入16-32位非空字符串' }}
22       </text>
23       <button class="save-btn pet-btn" :disabled="!isKeyValid"
24         @click="saveAccessKey">
25         保存密钥
26       </button>
27     </view>
28
29     <view class="help-info">

```

```
28     <text class="info-title">获取密钥方式</text>
29     <text class="info-content">1. 联系后端开发人员获取专属密钥</text>
30     <text class="info-content">2. 登录开发者平台自行申请</text>
31   </view>
32 </view>
33 </template>
34
35 <script setup>
36 import { ref, onLoad } from 'vue'
37 import accessKeyUtil from '../utils/accessKeyUtil'
38
39 // 页面数据
40 const accessKey = ref('')
41 const isKeyValid = ref(false)
42
43 // 页面加载时获取已存储的密钥
44 onLoad(() => {
45   const storedKey = accessKeyUtil.getAccessKey()
46   if (storedKey) {
47     accessKey.value = storedKey
48     validateInput(storedKey)
49   }
50 })
51
52 // 验证密钥输入格式
53 const validateInput = (val) => {
54   const value = val || accessKey.value
55   // 验证规则：非空且长度16-32位
56   isKeyValid.value = !!value && (value.length >= 16 && value.length <= 32)
57 }
58
59 // 保存密钥到本地
60 const saveAccessKey = () => {
61   const success = accessKeyUtil.setAccessKey(accessKey.value)
62   if (success) {
63     uni.showToast({ title: '密钥保存成功' })
64     // 保存成功后返回上一页
65     setTimeout(() => {
66       uni.navigateBack()
67     }, 1500)
68   } else {
69     uni.showToast({ title: '密钥保存失败，请重试', icon: 'error' })
70   }
71 }
72 </script>
73
74 <style lang="scss" scoped>
```

```
75 .access-key-page {
76   padding: 30rpx 24rpx;
77 }
78 .page-header {
79   margin-bottom: 40rpx;
80 }
81 .page-title {
82   font-size: 36rpx;
83   font-weight: bold;
84   color: var(--text-color);
85   display: block;
86   margin-bottom: 12rpx;
87 }
```

2. 业务页面调用示例（以首页为例）

原有业务页面的API调用逻辑无需修改，请求工具类会自动从本地获取access-key并注入请求头，实现无缝适配。

Code block

```
1  <script setup>
2  import { ref, onLoad } from 'vue'
3  import request from '../utils/request'
4
5  // 页面数据
6  const isLoading = ref(true)
7  const swiperList = ref([])
8  const recommendList = ref([])
9
10 // 页面加载时调用API
11 onLoad(async () => {
12   try {
13     // 调用首页API，请求工具类自动注入access-key
14     const homeData = await request.get('/home')
15     // 解构数据并渲染
16     swiperList.value = homeData.swiper || []
17     recommendList.value = homeData.recommend || []
18   } catch (err) {
19     console.error('首页数据加载失败: ', err)
20   } finally {
21     isLoading.value = false
22   }
23 })
24 </script>
```

3. 特殊接口：忽略access-key的调用

部分公开接口（如密钥申请指引接口）可能无需access-key，可通过 `ignoreAccessKey: true` 配置跳过密钥验证。

Code block

```
1  // 调用无需密钥的公开接口
2  const getKeyGuide = async () => {
3    try {
4      const guideData = await request.get('/public/access-key-guide', {}, {
5        ignoreAccessKey: true, // 跳过access-key验证
6        showLoading: false
7      })
8      console.log('密钥申请指引: ', guideData)
9    } catch (err) {
10     console.error('获取指引失败: ', err)
11   }
12 }
```

4. 页面级覆盖密钥

若某页面需要使用与全局不同的access-key，可在请求头中直接指定，优先级高于全局密钥。

Code block

```
1  // 页面级使用特殊access-key
2  const getSpecialData = async () => {
3    try {
4      const specialData = await request.get('/special/pet-list', {}, {
5        header: {
6          'access-key': 'SPECIAL_KEY_1234567890ABCDEF' // 覆盖全局密钥
7        }
8      })
9      console.log('特殊接口数据: ', specialData)
10     } catch (err) {
11       console.error('获取特殊数据失败: ', err)
12     }
13   }
```

五、多端适配与安全注意事项

1. 多端存储兼容性

- **小程序端**：uni.getStorageSync/setStorageSync在微信、支付宝等小程序中均支持，无需额外适配，但需注意小程序的存储容量限制（通常为10MB）。

- **App端**：支持本地存储，若需更高安全性，可结合plus.storage或加密存储（如使用crypto-js对密钥加密后再存储）。
- **H5端**：依赖浏览器localStorage，若浏览器禁用localStorage，会导致密钥存储失败，需在accessKeyUtil中添加异常捕获并提示用户开启。

2. 安全防护措施

- **禁止硬编码**：生产环境中需移除config.js中的defaultKey，通过用户输入或登录接口动态获取密钥，避免密钥泄露。
- **加密存储**：对存储的access-key进行加密处理（如AES加密），防止通过本地存储直接获取明文密钥。
- **接口权限控制**：配合服务端实现access-key的时效管理，定期更换密钥，降低密钥泄露风险。
- **错误屏蔽**：生产环境中避免在错误提示中暴露详细的密钥信息或接口地址，防止恶意攻击。

3. 异常场景处理

异常场景	处理逻辑	用户反馈
本地无access-key	拦截请求，跳转到密钥配置页	提示“密钥缺失，请先配置”
密钥格式错误	配置页实时验证，禁止保存	提示“请输入16-32位密钥”
服务端返回密钥无效	清除本地密钥，跳转到配置页	提示“密钥无效，请重新配置”
存储失败（如H5禁用localStorage）	捕获存储异常，提示用户开启存储权限	提示“存储权限被禁用，请开启后重试”

六、总结

本方案通过“配置文件+密钥工具类+请求工具类”的三层架构，实现了access-key在萌宠案例中的规范传递与管理，核心优势如下：

1. **低侵入性**：业务页面无需修改原有API调用逻辑，请求工具类自动完成密钥注入，实现无缝适配。
2. **完整的异常链路**：覆盖密钥缺失、格式错误、无效等全场景异常，提供清晰的用户引导与错误反馈。
3. **灵活可扩展**：支持全局密钥、页面级覆盖、忽略密钥等多种场景，适配不同接口的权限需求。
4. **多端兼容**：统一处理各平台存储差异，确保密钥在小程序、App、H5端均能正常存储与使用。

实际部署时，需结合服务端的access-key验证规则调整错误码与请求头键名，同时加强密钥的安全管理（如加密存储、定期更换），确保接口访问的安全性与稳定性。

(注：文档部分内容可能由 AI 生成)